

EyeDeal

Documentazione tecnica dettagliata delle funzioni e dei moduli dell'applicazione web ALUA EyeDeal, redatta a supporto della tesi di laurea sul progetto EyeDeal di ALUA.

Versione applicazione	0.0.0 (alua-app)
Stack tecnologico	React 18 · Vite 4 · Tailwind CSS 3 · react-pdf · react-zoom-pan-pinch
Tipo applicazione	Progressive Web App (PWA) — Single Page Application
Data documento	10/04/2026
Licenza	CC BY-NC-ND 4.0 — © 2025 ALUA Systems

Documento generato automaticamente dall'analisi del codice sorgente del repository *zjncoo/alua*. Tutti i riferimenti tecnici sono tratti direttamente dai file **src/App.jsx**, **index.html**, **package.json** e dalle risorse statiche della cartella **public/**.

Indice

- 1. Panoramica del Progetto**
 - 1.1 Contesto e scopo
 - 1.2 Architettura generale
- 2. Stack Tecnologico e Dipendenze**
 - 2.1 Runtime e build toolchain
 - 2.2 Librerie principali
 - 2.3 Configurazione PWA
- 3. Struttura dell'Applicazione**
 - 3.1 Entry point e DOM root
 - 3.2 Componente principale App
 - 3.3 Gestione degli stati (useState)
- 4. Flusso di Navigazione**
 - 4.1 Splash Screen
 - 4.2 Vista Login (Identificazione)
 - 4.3 Vista Dashboard
- 5. Acquisizione e Decodifica dei Dati Biometrici**
 - 5.1 Struttura del QR code
 - 5.2 Parsing dei parametri URL
 - 5.3 Retrocompatibilità con QR legacy
 - 5.4 Mapping clausole e tipi di relazione
- 6. Componenti UI — Funzioni Tecniche**
 - 6.1 LissajousFigure — Visualizzazione biometrica
 - 6.2 StoryTemplate — Condivisione social
 - 6.3 ContactCard — Schede team
- 7. Funzioni e Handler**
 - 7.1 handleLogin — Autenticazione sessione
 - 7.2 handleDisconnect — Reset sessione
 - 7.3 handleReport — Segnalazione violazione
 - 7.4 handleShare — Generazione immagine e condivisione
 - 7.5 handleCloseContract / handleCloseFullContract
 - 7.6 Helpers: toRoman, formatItalianText, getWeakLinkText
- 8. Visualizzatore PDF Integrato**
 - 8.1 react-pdf e pdfjs-dist
 - 8.2 Pinch-to-zoom con react-zoom-pan-pinch
- 9. Persistenza e Sessione**
 - 9.1 localStorage
 - 9.2 Gestione URL e history API
- 10. PWA — Progressive Web App**
 - 10.1 Manifest e icone
 - 10.2 Prompt di installazione
- 11. Dizionario dei Parametri URL**
- 12. Diagramma del Flusso Dati**
- 13. Glossario Tecnico**

Panoramica del Progetto

1.1 Contesto e scopo

ALUA EyeDeal è un'applicazione web sviluppata come interfaccia digitale della macchina fisica omonima. La macchina EyeDeal acquisisce dati biometrici da due persone tramite sensori di conduttanza cutanea (GSR — Galvanic Skin Response) e genera un *contratto digitale di relazione* che esprime, in forma quasi-legale e visiva, la compatibilità emotiva e psicofisiologica rilevata durante la sessione.

L'applicazione web ha il ruolo di: (a) ricevere i dati prodotti dalla macchina tramite un QR code generato al termine della sessione, (b) ricostruire e visualizzare il contratto digitale, (c) permettere ai contraenti di consultare le clausole, il documento PDF legale, la fascia di rischio e il grado di affinità, e (d) segnalare eventuali violazioni contrattuali tramite un bottone dedicato.

Il progetto si inserisce nell'ambito dell'arte relazionale e della critica ai sistemi assicurativi e contrattuali moderni: il linguaggio burocratico e le metriche quantitative vengono applicati alle relazioni umane, creando un corto circuito semantico che invita alla riflessione.

1.2 Architettura generale

L'applicazione è una **Single Page Application (SPA)** React che funziona come **Progressive Web App (PWA)**. Non dispone di backend proprio: tutti i dati del contratto viaggiano codificati nel QR code come parametri querystring dell'URL. La lettura avviene interamente lato client.

Livello	Tecnologia	Responsabilità
Presentazione	React 18 + JSX	Rendering UI, gestione stati, animazioni
Stile	Tailwind CSS 3	Layout responsive, tipografia, colori brand
Build	Vite 4	Bundling, HMR, ottimizzazione asset
PDF viewer	react-pdf 10 + pdfjs-dist	Rendering documenti PDF nel browser
Zoom/Pan	react-zoom-pan-pinch 3	Interazione pinch-to-zoom su mobile
Icone	lucide-react 0.263	Icone SVG vettoriali inline
Screenshot	html2canvas 1.4 (CDN)	Cattura canvas per condivisione social
Dati	URL querystring (QR)	Trasporto dati biometrici dalla macchina
Persistenza	localStorage	Nomi contraenti tra sessioni

Stack Tecnologico e Dipendenze

2.1 Runtime e build toolchain

Il progetto viene avviato con **Vite 4**, un bundler moderno basato su ESModules che offre un server di sviluppo con Hot Module Replacement (HMR) quasi istantaneo e un processo di build ottimizzato tramite Rollup. Il file di configurazione è **vite.config.js** con il plugin **@vitejs/plugin-react** che abilita il supporto a JSX e Fast Refresh per React.

Tailwind CSS viene processato tramite PostCSS (file **postcss.config.js**) con il plugin **autoprefixer**: in fase di build genera solo le classi effettivamente utilizzate (tree-shaking CSS), minimizzando il bundle finale.

2.2 Librerie principali

react ^18.2.0

Libreria core per la creazione di interfacce utente basata su componenti. Usa il reconciler virtuale DOM per aggiornamenti efficienti.

react-dom ^18.2.0

Renderer React per il browser. Espone ReactDOM.createRoot() usato in main.jsx per montare l'app sul div#root.

lucide-react ^0.263.1

Set di icone SVG come componenti React. Importate singolarmente (tree-shakable) per minimizzare il bundle.

react-pdf ^10.3.0

Visualizzatore PDF basato su pdfjs-dist. Espone i componenti e che rendono ogni pagina su canvas HTML5.

react-zoom-pan-pinch ^3.7.0

Libreria per gesture di zoom/pan/pinch su mobile. Usata per avvolgere il visualizzatore PDF e abilitare il pinch-to-zoom.

html2canvas 1.4 (CDN)

Libreria per catturare elementi DOM come immagine canvas. Caricata da CDN in index.html per generare l'immagine da condividere.

pdfjs-dist (worker)

Worker JavaScript per il rendering PDF in un thread separato. Configurato via pdfjs.GlobalWorkerOptions.workerSrc puntando a unpkg CDN.

2.3 Configurazione PWA

L'app è configurata come Progressive Web App tramite i meta tag nel file **index.html**. I tag rilevanti sono:

meta apple-mobile-web-app-capable	Abilita la modalità fullscreen su iOS quando aggiunta alla Home Screen.
meta apple-mobile-web-app-status-bar-style	Imposta il colore della status bar su iOS.
meta theme-color	Imposta il colore della barra del browser su Android Chrome.
link apple-touch-icon	Icona PNG 180×180 per il launcher iOS.
link icon SVG	Favicon SVG per browser desktop.

Struttura dell'Applicazione

3.1 Entry point e DOM root

Il file `src/main.jsx` è il punto di ingresso dell'applicazione. Usa `ReactDOM.createRoot(document.getElementById('root'))` per montare il componente `<App />` all'interno del `<div id="root">` presente in `index.html`. Il wrapping con `<React.StrictMode>` attiva i controlli aggiuntivi di React in fase di sviluppo (doppie invocazioni degli effect, warning di deprecazioni).

3.2 Componente principale App

Tutto il codice dell'interfaccia è contenuto nel singolo file `src/App.jsx` (~1560 righe). Il file esporta il componente di default App e definisce internamente i seguenti sotto-componenti:

LissajousFigure	Disegna su canvas una figura di Lissajous basata sui parametri biometrici GSR.
StoryTemplate	Template HTML 1080×1920px nascosto, usato da html2canvas per generare l'immagine da condividere.
ContactCard	Modal che mostra la scheda personale di un membro del team quando il QR contiene il parametro <code>?contact=</code> .
App	Componente root che gestisce tutta la logica di navigazione, parsing URL, stati e rendering.

3.3 Gestione degli stati (useState)

L'applicazione utilizza esclusivamente hook React per la gestione dello stato, senza Redux o altri state manager esterni. Gli stati principali sono:

<code>showSplash / splashFading / contentVisible</code>	Gestione animazione splash screen iniziale (fade-out).
<code>view</code>	Navigazione tra le viste: 'LOGIN' o 'DASHBOARD'.
<code>systemStatus</code>	Stato del sistema di segnalazione: 'MONITORING' o 'REPORTED'.
<code>isPressed</code>	Flag animazione bottone segnalazione.
<code>showContract / isClosingContract / isOpeningContract</code>	Visibilità e animazione del modal contratto digitale.
<code>showFullContract / isClosingFullContract / isOpeningFullContract</code>	Visibilità e animazione del modal PDF.

numPages / containerWidth / pdfContainerRef	Dati per il rendering PDF adattivo.
time	Data/ora corrente, aggiornata ogni 1 secondo via setInterval.
showInstallPrompt / doNotShowAgain	Prompt di installazione PWA e preferenza utente.
partyA / partyB	Nomi dei due contraenti, provenienti da URL o input manuale.
contractData	Oggetto con tutti i dati del contratto (id, data, compatibilità, fascia rischio, ecc.).
debugParams	Tutti i parametri URL raw, per debug.
missingData	Flag che indica l'assenza di dati biometrici nel QR.
scannedMember	Dati del membro del team se il QR è una card personale.

Flusso di Navigazione

4.1 Splash Screen

Al primo caricamento, se l'URL non contiene il parametro `?contact=`, viene mostrata la splash screen: uno schermo bianco con il logo ALUA centrato. Tecnicamente lo stato `showSplash` è inizializzato a `true` (con una `lazy initializer function` che controlla i parametri URL).

Dopo 1000ms un `setTimeout` attiva `setSplashFading(true)` che aggiunge la classe CSS `opacity-0` con una transizione di 300ms. In contemporanea `setContentVisible(true)` avvia il fade-in del contenuto sottostante. Dopo 300ms, `setShowSplash(false)` rimuove definitivamente il componente dal DOM.

4.2 Vista Login (Identificazione)

La vista LOGIN è il punto di ingresso dopo lo splash. Mostra:

- Il logo ALUA collegato al sito istituzionale.
- Un pannello 'Dati Polizza' con l'ID contratto e la percentuale di affinità letti dall'URL.
- Due input testuali per inserire i nomi di Contraente A e Contraente B.
- Il bottone 'Inizializza Sessione' che esegue `handleLogin()`.
- Il prompt di installazione PWA (se l'app non è già installata e non è stato dismesso).
- Il modal `ContactCard` se il QR è una scheda personale (`?contact=`).

Se i nomi sono già salvati nel `localStorage` (chiavi `alua_partyA` e `alua_partyB`), oppure se sono passati come parametri URL (`?partyA_name=&partyB_name=`), la navigazione salta direttamente alla Dashboard.

4.3 Vista Dashboard

La Dashboard è la schermata principale. È composta da:

Header fisso (sticky)	Mostra data, ora aggiornata al minuto, ID contratto, stato sistema (pallino animato) e logo ALUA.
Sezione Lissajous + Frase	Visualizzazione della curva biometrica e della frase dedicata del contratto, visibili solo se il parametro <code>?phrase=</code> è presente nell'URL.
Info Card	Pannello con ID contratto, bottone per aprire il modal contratto, nomi contraenti, eventuale alert di instabilità (weak link), e la grande visualizzazione della compatibilità percentuale e fascia di rischio.

Zona Azione (Report)	Area centrale con il grande bottone rosso circolare per segnalare una violazione contrattuale.
Footer fisso	Bottone 'Reset Sessione', dati di certificazione digitale, note legali fittizie di 'Alua S.p.A.' e copyright.

Acquisizione e Decodifica dei Dati Biometrici

5.1 Struttura del QR code

La macchina fisica EyeDeal (basata su microcontrollore Arduino e script Python `monitor_arduino.py` / `contract_generator.py`) al termine della sessione biometrica genera un QR code che punta all'URL dell'applicazione web con i dati del contratto codificati come parametri `querystring`.

Esempio di URL generato dalla macchina:

```
https://alua.app/?id=ABC-123&date=10.04.25&comp=78&fascia=2&cost=45,00€&bad=1&avg0=312.4&avg1=298.7&types=ROMANTICA,AMICALE&phrase=Due+anime+che+si+riconoscono&mid=ALUA-M-V1&partyA_name=MARCO&partyB_name=SARA
```

5.2 Parsing dei parametri URL

Il parsing avviene all'interno dell'hook `useEffect([], [])` (eseguito una sola volta al montaggio del componente) tramite l'API nativa del browser `URLSearchParams`:

Parametro URL	Tipo	Descrizione	Default
id	string	Identificativo univoco del contratto	UNK-00
date	string	Data di stipula (formato DD.MM.YY)	data corrente +2 anni
comp	integer	Percentuale di compatibilità (0–100)	50
fascia	integer	Fascia di rischio (1–5)	1
cost	string	Premio annuo del contratto	0,00€
bad	integer	Weak link: -1=nessuno, 0=A, 1=B	-1
avg0	float	Valore SCL medio del Contraente A	0
avg1	float	Valore SCL medio del Contraente B	0
types	string	Tipi di relazione separati da virgola	—
phrase	string	Frase dedicata del contratto	—
mid	string	ID macchina che ha generato il contratto	ALUA-M-V1
partyA_name	string	Nome pre-compilato Contraente A	—
partyB_name	string	Nome pre-compilato Contraente B	—
contact	string	Nome membro team (QR card personale)	—

5.3 Retrocompatibilità con QR legacy

I QR code generati da versioni precedenti della macchina usavano indici numerici invece dei nomi delle relazioni (`?btn0=0,2&btn1=1` invece di `?types=CONOSCENZA,LAVORATIVA,ROMANTICA`). Il codice mantiene una logica di fallback:

Se `params.get('types')` è vuoto, vengono letti i parametri legacy `btn0` e `btn1` come array di indici numerici. Questi indici vengono convertiti in chiavi testuali tramite l'array `RELAZIONI_KEYS` `['CONOSCENZA', 'ROMANTICA', 'LAVORATIVA', 'AMICALE', 'FAMILIARE', 'CONVIVENZA']`, dove l'indice corrisponde alla posizione del bottone fisico sulla macchina Arduino. I duplicati vengono rimossi con `Set`.

5.4 Mapping clausole e tipi di relazione

L'oggetto `CLAUSE_MAPPING` associa ogni tipo di relazione al testo della clausola corrispondente, usando spazi non-breaking (`\u00A0`) per preservare la formattazione tipografica:

Chiave (tipo relazione)	Testo clausola
CONOSCENZA	Esplorazione preliminare.
ROMANTICA	Tensione attrattiva e vulnerabilità emotiva.
LAVORATIVA	Collaborazione formale, efficienza prioritaria.
AMICALE	Supporto reciproco, tempo non strutturato.
FAMILIARE	Legame di appartenenza e obblighi impliciti.
CONVIVENZA	Condivisione di spazi riservati.

Componenti UI — Funzioni Tecniche

6.1 LissajousFigure — Visualizzazione biometrica

Il componente LissajousFigure è il cuore della visualizzazione biometrica. Riceve tre props:

gsr0	Valore SCL medio del Contraente A (float).
gsr1	Valore SCL medio del Contraente B (float).
compatibility	Percentuale di compatibilità 0–100 (integer).

Algoritmo di rendering (Canvas 2D API):

Il componente usa useRef per ottenere un riferimento diretto all'elemento <canvas> (256×256 px) e useEffect per ridisegnare la figura ogni volta che cambiano le props.

- **Calcolo val_gsr:** media aritmetica di gsr0 e gsr1.
- **freq_x:** $1 + \text{floor}((\text{val_gsr} / 1000.0) \times 12)$. Valori GSR più alti producono figure più complesse (più 'nodi').
- **freq_y:** freq_x + 1 (se uguale a freq_x viene incrementato ulteriormente per evitare una linea retta).
- **delta:** $(\text{compatibility} / 100.0) \times \pi$. Compatibilità alta → figura più simmetrica e 'armonica'.
- **Disegno:** per t da 0 a 2π (1000 passi): $x = cx + r \cdot \sin(\text{freq_x} \cdot t + \text{delta})$, $y = cy - r \cdot \sin(\text{freq_y} \cdot t)$. Il canvas viene pulito con clearRect, poi si tracciano le linee con beginPath/lineTo/stroke.

Il risultato è una curva parametrica unica per ogni coppia di dati biometrici: cambia forma al variare del livello di attivazione GSR e del grado di compatibilità, rendendo ogni contratto visivamente irripetibile.

6.2 StoryTemplate — Condivisione social

Il componente StoryTemplate genera un layout HTML statico in formato 1080×1920px (formato Instagram Stories), posizionato fuori dalla viewport (left: -9999px; top: -9999px) per essere invisibile all'utente ma accessibile al DOM per html2canvas.

Il template include: intestazione con logo ALUA, nomi dei contraenti, percentuale di compatibilità in grande (150px), fascia di rischio in numeri romani (funzione toRoman()), figura di Lissajous in tempo reale, frase dedicata del contratto formattata con formatItalianText(), e footer con dati tecnici.

Perché pointerEvents: none invece di visibility: hidden? html2canvas ha problemi con elementi impostati come hidden o display: none: non riesce a catturarne il contenuto. Usando pointerEvents: none l'elemento è renderizzato normalmente nel DOM ma non intercetta interazioni utente.

6.3 ContactCard — Schede team

Quando un QR code contiene il parametro `?contact=NomeCognome`, l'applicazione mostra un modal con la scheda personale del membro del team. La ricerca nel dizionario `GROUP_MEMBERS` è *case-insensitive* (entrambi i testi vengono convertiti in lowercase prima del confronto).

Il dizionario `GROUP_MEMBERS` contiene 6 voci corrispondenti ai membri del team ALUA, con profilo Instagram, URL e percorso immagine:

Nome	Instagram	Immagine
Camilla Costato	@ca.lcutta	/mugshots/CamillaCostato.webp
Francesco Zanchetta	@zjncoo	/mugshots/FrancescoZanchetta.webp
Camilla Vicario	@camiivicario	/mugshots/CamillaVicario.webp
Davide Benetton	@davidebenetton_	/mugshots/DavideBenetton.webp
Sofia Splendore	@sofia.splendore	/mugshots/SofiaSplendore.webp
Alice D'Andrea	@alicee.dandrea	/mugshots/AliceDAndrea.webp

Funzioni e Handler

7.1 handleLogin — Autenticazione sessione

Gestore del submit del form nella vista Login. Funzionamento:

- **preventDefault()** blocca il reload della pagina.
- Se entrambi partyA e partyB sono non vuoti, salva i nomi in localStorage.
- Chiama navigator.vibrate(50) per feedback aptico (se supportato).
- Imposta view = 'DASHBOARD' per navigare alla dashboard.

7.2 handleDisconnect — Reset sessione

Chiamato dal bottone 'Reset Sessione' nel footer della dashboard:

- Rimuove le chiavi alua_partyA e alua_partyB dal localStorage.
- Resetta gli stati partyA, partyB a stringa vuota.
- Imposta view = 'LOGIN'.
- Resetta systemStatus = 'MONITORING'.

7.3 handleReport — Segnalazione violazione

Il cuore narrativo dell'applicazione: permette di segnalare una violazione del contratto relazionale. Funzionamento:

- Imposta isPressed = true (feedback visivo di pressione).
- Vibrazione lunga 200ms via navigator.vibrate(200).
- Dopo 800ms: imposta systemStatus = 'REPORTED' e disabilita il bottone.
- Tripla vibrazione breve [50, 50, 50]ms come conferma segnalazione.
- Il bottone rosso cambia colore da #C3000C (rosso ALUA) a grigio chiaro.
- Appare un messaggio 'Segnalazione Registrata' con CheckCircle icon.
- Un bottone 'Invia Nuova Segnalazione' permette di resettare tramite resetSystem().

7.4 handleShare — Generazione immagine e condivisione

Questa funzione implementa la condivisione del contratto come immagine per Instagram Stories. Il processo è il più complesso dell'applicazione:

1. Vibrazione aptica (50ms) come feedback di avvio.
2. Recupera l'elemento DOM #share-story-template (il componente StoryTemplate nascosto).

3. Chiama `window.html2canvas(element, options)` con `scale 2x` per qualità Retina, `useCORS: true`, dimensioni `1080×1920`, e un hook `onclone` che forza il `display` dell'elemento nel documento clonato (risolve bug di `visibility`).
4. `html2canvas` ritorna un oggetto `canvas`. Viene chiamato `canvas.toBlob()` per ottenere un Blob PNG.
5. Il Blob viene wrappato in un oggetto `File` con nome `ALUA_Story_{id}.png`.
6. Se `navigator.share` è disponibile (Web Share API, supportata su iOS/Android), viene usata per aprire il pannello di condivisione nativo del sistema operativo.
7. Se la Web Share API non è disponibile (es. desktop), viene generato un link `<a download>` e triggerato con `link.click()` per scaricare l'immagine.

7.5 `handleCloseContract / handleCloseFullContract`

Entrambe le funzioni implementano la stessa logica di chiusura animata (slide-down):

- Impostano `isClosing* = true` che aggiunge `translate-y-full` via Tailwind, animando il modal verso il basso.
- Dopo 500ms (durata dell'animazione CSS), impostano `show* = false` per rimuovere il componente dal DOM e `isClosing* = false` per resettare lo stato.

L'apertura del modal funziona in modo speculare: `show* = true` monta il componente con `translate-y-full` (invisibile in basso), poi `requestAnimationFrame()` assicura che il browser abbia eseguito il primo render prima di impostare `isOpening* = true` che porta `translate-y-0`, producendo l'animazione slide-up.

7.6 Helpers: `toRoman`, `formatItalianText`, `getWeakLinkText`

toRoman(num)

Converte un intero in numerale romano tramite una tabella di lookup ordinata dal valore maggiore (M=1000) al minore (I=1). Iterando sulla tabella e sottraendo ripetutamente il valore corrente finché `num >= lookup[i]`, costruisce il simbolo romano corrispondente. Usato per visualizzare la fascia di rischio nella `StoryTemplate`.

formatItalianText(text)

Applica le convenzioni editoriali italiane al testo delle frasi del contratto. Sostituisce lo spazio normale con lo spazio non-interrompibile (U+00A0) dopo articoli, preposizioni, congiunzioni e avverbi comuni (es. 'il', 'di', 'e', 'non'). Previene anche la separazione prima della punteggiatura. Infine inserisce un a-capo dopo ogni punto fermo, punto esclamativo e interrogativo per migliorare la leggibilità nel layout.

getWeakLinkText()

Ritorna un messaggio di avviso se `contractData.weakLink` è 0 ('Instabilità rilevata in CONTRAENTE A') o 1 ('...in CONTRAENTE B'), altrimenti null. Il valore proviene dal parametro URL `?bad=` generato dalla macchina quando uno dei due contraenti mostra un livello di stress GSR significativamente superiore all'altro.

Visualizzatore PDF Integrato

8.1 react-pdf e pdfjs-dist

L'applicazione integra un visualizzatore PDF completo basato sulla libreria **react-pdf v10**, che a sua volta usa **pdfjs-dist** (la distribuzione npm di PDF.js, il renderer PDF open-source di Mozilla).

Il worker di rendering è configurato puntando alla CDN unpkg:

```
pdfjs.GlobalWorkerOptions.workerSrc =  
`//unpkg.com/pdfjs-dist@${pdfjs.version}/build/pdf.worker.min.mjs`
```

Il file PDF del contratto (**/contratto.pdf** nella cartella public) viene caricato dal componente `<Document file='/contratto.pdf'>`. Ogni pagina viene renderizzata con il componente `<Page>` con larghezza adattiva (`containerWidth - 32px`) calcolata tramite un `useEffect` che misura `pdfContainerRef.current.clientWidth` e aggiorna il valore al `resize` della finestra.

L'importazione dei CSS layer `react-pdf/dist/Page/AnnotationLayer.css` e `TextLayer.css` abilita rispettivamente i link cliccabili nelle annotazioni PDF e la selezione del testo nelle pagine.

Il parametro `devicePixelRatio={window.devicePixelRatio || 2}` assicura la massima nitidezza su schermi Retina/HiDPI.

8.2 Pinch-to-zoom con react-zoom-pan-pinch

Per garantire una buona esperienza mobile, il visualizzatore PDF è avvolto dai componenti `<TransformWrapper>` e `<TransformComponent>` di **react-zoom-pan-pinch**.

<code>initialScale</code>	1 (nessuno zoom al caricamento)
<code>minScale</code>	0.5 (massimo rimpicciolimento 50%)
<code>maxScale</code>	3 (massimo ingrandimento 300%)
<code>wheel.disabled</code>	true (scroll mouse non fa zoom per non interferire con lo scroll verticale)
<code>pinch.step</code>	5 (sensibilità gesto pinch)
<code>doubleClick.mode</code>	'reset' (doppio tap resetta lo zoom a 1x)
<code>panning.disabled</code>	false (trascinamento abilitato)

Persistenza e Sessione

9.1 localStorage

L'applicazione usa l'API localStorage del browser per persistere i nomi dei contraenti tra sessioni diverse:

Chiave localStorage	Valore	Quando viene scritta	Quando viene letta
alua_partyA	Nome Contraente A (upload login) o parametri URL	Avvio app (useEffect)	
alua_partyB	Nome Contraente B (upload login) o parametri URL	Avvio app (useEffect)	
alua_install_prompt_seen	'true'	handleDismissInstall() con check	Avvio app (useEffect PWA)

Grazie al localStorage, se due persone hanno già effettuato il login e riaprono l'app dallo stesso dispositivo (es. dopo aver chiuso il browser), saltano direttamente alla Dashboard senza dover reinserire i nomi.

9.2 Gestione URL e history API

Quando viene mostrata una ContactCard (QR personale) e l'utente la chiude, l'URL viene ripulito dal parametro ?contact= tramite `window.history.replaceState({}, "", window.location.pathname)`. Questo evita che, ricaricando la pagina, la card riappaia.

PWA — Progressive Web App

10.1 Manifest e icone

L'applicazione è configurata come PWA tramite meta tag HTML (non tramite un file manifest.json separato). Le icone sono servite dalla cartella public/:

- **logo_alua.svg** — Favicon SVG per browser desktop.
- **icon.png** (512×512, 180×180) — Icona PNG per iOS e Android.
- **logo_alua.png** — Usata nel template StoryTemplate.

La viewport è configurata con maximum-scale=1.0, user-scalable=no per prevenire lo zoom automatico iOS sui campi di input (che avviene quando il font size è <16px).

10.2 Prompt di installazione

Il componente mostra un banner di installazione agli utenti non-PWA. La logica è implementata in un useEffect dedicato che:

- Controlla window.matchMedia('(display-mode: standalone)').matches e window.navigator.standalone per rilevare se l'app è già installata.
- Controlla il flag localStorage alua_install_prompt_seen.
- Se non installata e non dismessa, mostra il banner dopo 2 secondi (setTimeout).
- Il banner mostra istruzioni passo-passo per iOS (Condividi → Aggiungi alla Home).
- La checkbox 'Non mostrare più' salva il flag nel localStorage.

Dizionario dei Parametri URL

Riferimento completo di tutti i parametri URL che l'applicazione è in grado di leggere, con tipo, valori ammissibili e provenienza:

Param	Tipo	Valori	Provenienza	Comportamento se assente
id	string	Alfanumerico (es. ABC-123)	machine	Default 'UNK-00'
date	string	DD.MM.YY	machine	Data corrente +2 anni
comp	int 0-100	0–100	machine (GSR)	Default 50; flag missingData=true
fascia	int 1-5	1,2,3,4,5	machine	Default 1
cost	string	Prezzo (es. 45,00€)	machine	Default '0,00€'
bad	int	-1, 0, 1	machine (GSR)	Default -1 (nessun weak link)
avg0	float	Valore SCL	machine (GSR)	Default 0
avg1	float	Valore SCL	machine (GSR)	Default 0
types	string	CONOSCENZA ROMANTICA AMICALE FAMILIARE ACQUAINTANZA CONOSCENZA	machine (GSR)	Default 'CONOSCENZA'
phrase	string	Testo libero (URL-encoded)	machine (AI)	Nessuna frase visualizzata
mid	string	Alfanumerico	machine	Default 'ALUA-M-V1'
partyA_name	string	Nome (uppercase)	machine o URL manuale	Login manuale richiesto
partyB_name	string	Nome (uppercase)	machine o URL manuale	Login manuale richiesto
btn0	string	Indici CSV (es. 0,2)	machine legacy	Ignorato se types presente
btn1	string	Indici CSV (es. 1)	machine legacy	Ignorato se types presente
contact	string	Nome membro team	QR card personale	Nessuna ContactCard

Diagramma del Flusso Dati

Il seguente schema descrive il flusso completo dei dati dall'acquisizione biometrica alla visualizzazione nell'app:

Step	Attore/Sistema	Descrizione
1	Sensori GSR (Arduino)	Misurazione conduttanza cutanea (SCL) per entrambi i contraenti durante la sessione.
2	monitor_arduino.py	Script Python che legge i dati seriali dall'Arduino, calcola compatibilità, fascia rischio, individua il w
3	contract_generator.py	Genera il PDF del contratto fisico e costruisce la querystring con tutti i parametri.
4	QR Code generator	Codifica l'URL completo in un QR code stampato/mostrato ai contraenti.
5	Browser (scan QR)	L'utente scansiona il QR con il proprio smartphone. Il browser apre l'URL dell'app web.
6	React useEffect (parsing)	Al montaggio, URLSearchParams legge tutti i parametri dall'URL e popola contractData.
7	Vista Login	Se i nomi non sono nell'URL o localStorage, l'utente li inserisce manualmente.
8	Vista Dashboard	Visualizzazione completa: Lissajous, frase, affinità, fascia rischio, modal contratto.
9	handleShare	html2canvas cattura il template nascosto e genera l'immagine Instagram Story (1080×1920px).
10	Web Share API	Condivisione nativa iOS/Android oppure download diretto su desktop.

In sintesi: la macchina EyeDeal agisce da *data producer*, codificando l'intero output della sessione biometrica in una singola stringa URL. L'app web agisce da *data consumer e renderer* stateless: non ha bisogno di alcun backend o database perché tutti i dati necessari sono già nell'URL.

Glossario Tecnico

GSR / SCL

Galvanic Skin Response / Skin Conductance Level. Misura della conduttanza elettrica della pelle, correlata all'attivazione del sistema nervoso autonomo (stress, eccitazione emotiva). Viene misurata in microsiemens (μS).

Lissajous (figura di)

Curva parametrica bidimensionale $x(t) = A \cdot \sin(a \cdot t + \delta)$, $y(t) = B \cdot \sin(b \cdot t)$. La forma dipende dal rapporto a/b delle frequenze e dalla fase δ . Nell'app, le frequenze derivano dal valore GSR e la fase dalla compatibilità.

QR Code

Quick Response Code. Codice a barre bidimensionale che in questo contesto codifica un URL con parametri querystring contenenti i dati biometrici del contratto.

PWA

Progressive Web App. Applicazione web che può essere installata come app nativa, funziona offline e si integra con il sistema operativo (splash screen, icona Home, notifiche).

SPA

Single Page Application. Architettura web dove la navigazione avviene senza ricaricare la pagina, tramite aggiornamenti dinamici del DOM gestiti da JavaScript.

URLSearchParams

API nativa del browser per il parsing e la manipolazione dei parametri querystring di un URL.

localStorage

API del browser per la persistenza di dati chiave-valore nel browser dell'utente. I dati sopravvivono alla chiusura del browser (a differenza di sessionStorage).

html2canvas

Libreria JavaScript che 'fotografa' un elemento DOM trasformandolo in un canvas HTML5, permettendo l'esportazione come immagine PNG/JPEG.

Web Share API

API del browser che delega la condivisione di file e testi al sistema operativo nativo (foglio di condivisione iOS/Android).

react-pdf / pdfjs

react-pdf è un wrapper React per pdfjs-dist (PDF.js di Mozilla), che renderizza documenti PDF pagina per pagina su canvas HTML5.

react-zoom-pan-pinch

Libreria React per aggiungere gesture di zoom (pinch), pan e rotazione a qualsiasi elemento DOM.

Fascia di rischio

Valore 1–5 generato dalla macchina che esprime il 'rischio' della relazione in termini pseudo-assicurativi. Deriva dall'analisi delle variazioni GSR.

Weak Link

Il contraente che mostra maggiore instabilità emotiva (GSR più variabile) durante la sessione. Identificato dal parametro ?bad= (0=A, 1=B, -1=nessuno).

Compatibilità

Percentuale 0–100 calcolata dalla macchina sulla base della correlazione tra le risposte GSR dei due contraenti.

requestAnimationFrame

API del browser che esegue una callback prima del prossimo frame di rendering. Usata per sincronizzare le animazioni CSS con il ciclo di rendering del browser.

useEffect / useState / useRef / useCallback

Hook fondamentali di React: useEffect per effetti collaterali, useState per lo stato locale, useRef per riferimenti DOM, useCallback per memoizzare funzioni.

ALUA SYSTEMS

Questo documento è stato generato automaticamente dall'analisi del codice sorgente del repository *zjncoo/alua* e costituisce la documentazione tecnica ufficiale dell'applicazione web EyeDeal a supporto delle attività di ricerca e divulgazione accademica del progetto ALUA.

Documento generato il 10/04/2026 — © 2025 ALUA Systems — CC BY-NC-ND 4.0